

HW8

STAT 651

RJ Cass

2026-04-03

1

With the same fit (Hoff 8.3) from HW7, do the following:

a)

Perform posterior check with $T(y, \theta) = \max_j (Var(y_{i,j})) - \min_j (Var(y_{i,j}))$

STAN code for parts (a)-(e)

```
data {  
  int<lower=0> N; //Number of Schools  
  int<lower=0> n; //Total datapoints  
  array[n] real y; // Responses  
  array[n] int grp; //Associated group for each response  
  real nu_0;  
  real sigma2_0;  
  real eta_0;  
  real tau2_0;  
  real mu_0;  
  real gamma2_0;  
}  
  
parameters {  
  array[N] real theta;  
  real mu;  
  real<lower=0> sigma2;  
  real<lower=0> tau2;  
}  
  
transformed parameters {  
  array[n] real theta_vec;
```

```

    for (i in 1:n) {
      theta_vec[i] = theta[grp[i]];
    }
  }

model {
  y ~ normal(theta_vec, sqrt(sigma2));
  theta ~ normal(mu, sqrt(tau2));
  sigma2 ~ inv_gamma(nu_0/2, nu_0 * sigma2_0/2);
  tau2 ~ inv_gamma(eta_0/2, eta_0 * tau2_0/2);
  mu ~ normal(mu_0, sqrt(gamma2_0));
}

generated quantities {
  vector[n] y_rep;
  vector[n] y_upc;
  vector[n] y_llik;
  vector[N] theta_upc;

  for (i in 1:n) {
    real theta_i = theta[grp[i]];
    y_rep[i] = normal_rng(theta_i, sqrt(sigma2));
    y_upc[i] = normal_cdf(y[i] | theta_i, sqrt(sigma2));
    y_llik[i] = normal_lpdf(y[i] | theta_i, sqrt(sigma2));
  }

  for (i in 1:N) {
    theta_upc[i] = normal_cdf(theta[i] | mu, tau2);
  }
}

```

```

suppressPackageStartupMessages(library("rstan"))
suppressPackageStartupMessages(library("goftest"))
suppressPackageStartupMessages(library("loo"))
suppressPackageStartupMessages(library("knitr"))
dt <- read.csv('schools651.csv', header = TRUE)
dt <- dt[, -1] # Remove the rownames
mu_0 <- 7
gamma2_0 <- 5
tau2_0 <- 10
eta_0 <- 2
sigma2_0 <- 15
nu_0 <- 2
N <- 8 # Number of schools
n <- length(dt$school)

data_stan <- list(N = N, n = n, y = dt$times, grp = dt$school, nu_0 = nu_0, sigma2_0 = sigma2_0,

```

```

params <- c("theta", "mu", "sigma2", "tau2", "y_rep", "y_upc", "y_llik", "theta_upc")

n_cores <- parallel::detectCores() # Count how many cores are available
options(mc.cores = n_cores)

fit_stan <- stan(model_code = readLines("q1_a_e.stan"),
                data = data_stan, pars = params,
                iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan <- extract(fit_stan) # collect samples from all chains

n_sims <- nrow(sim_stan$theta)
var_diffs <- numeric(n_sims)
for (i in 1:n_sims) {
  var_diffs[[i]] <- diff(range(tapply(sim_stan$y_rep[i, ], dt$school, var)))
}

obs_var_diff <- diff(range(tapply(dt$times, dt$school, var)))
var_diff_p <- mean(var_diffs > obs_var_diff)

```

If the p-value for the range between the smallest and largest variances is large, that would imply that our assumption of shared variance between all the groups is not a reasonable assumption. If the p-value is small, then that would indicate that it is likely a reasonable assumption. A large p-value would imply that we should refit the model using individual group variances, as opposed to one joint population variance.

In this case, the range is 0.6325556.

b)

I propose a test statistic of the variance of the means: $T(y, \theta) = \text{Var}(\text{mean}_j(y_{i,j}))$. If this p-value is small, that could be an indication that we actually don't need to the different school means, and could instead use a shared mean across all.

```

means_var <- numeric(n_sims)
for (i in 1:n_sims) {
  means_var[[i]] <- var(tapply(sim_stan$y_rep[i, ], dt$school, mean))
}

obs_means_var <- var(tapply(dt$times, dt$school, mean))
means_var_p <- mean(means_var > obs_means_var)

```

This has a value of 0.6168889 meaning that our assumption of different means is probably a good assumption.

c)

i)

Single draw UPC test:

```
indx_upc <- 2000 # pick just one iteration of MCMC
test <- goftest::ad.test(sim_stan$y_upc[indx_upc,], null = "punif")
p_val <- round(test$p.value, 4)
```

This has a p-value of 0.3997, which seems to be a good value showing uniformity.

ii)

Combining p-values of uniformity tests across all posterior samples:

```
upc_pvals_y <- apply(sim_stan$y_upc, 1, function(x) goftest::ad.test(x, null = "punif")$p)
cauchy_y_p <- round(pcauchy(mean(tan(pi * (0.5 - upc_pvals_y))), lower.tail = FALSE), 4)
```

This gives a p-value of 0.1609 which would imply that the model is perhaps inadequately fit.

d)

```
upc_pvals_theta <- apply(sim_stan$theta_upc, 1, function(x) goftest::ad.test(x, null = "punif")$p)
cauchy_theta_p <- round(pcauchy(mean(tan(pi * (0.5 - upc_pvals_theta))), lower.tail = FALSE), 4)
```

Performing the UPC test for all the thetas gives the following p-value: 0.2901, which implies that our model does capture the behavior of the thetas reasonably well.

e)

```
loo1 <- loo::loo(sim_stan$y_lik)
loo1_est <- round(loo1$estimates[1, 1], 3)
loo1_params <- round(loo1$estimates[2, 1], 2)
waic1 <- loo::waic(sim_stan$y_lik)
waic1_est <- round(waic1$estimates[1, 1], 3)
waic1_params <- round(waic1$estimates[2, 1], 2)
```

This model gives a LOO of -499.358 and a WAIC of -499.337. The models give model parameter estimates of 7.64 (LOO) and 7.62 (WAIC).

These metrics give a lower number of estimated parameters than what we've used. This likely means that some of our parameters are extra, such as maybe some of the schools have similar/shared means, etc.

f)

I'm just going to let all the σ_j^2 be distributed as described in Hoff:

$\frac{1}{\sigma_j^2} \sim \text{Gamma}(\frac{\nu}{2}, \frac{\nu\sigma_0^2}{2})$, but I will pick $\nu \sim \text{Exp}(\alpha_0)$ where α_0 is a parameter we pick, to enforce that it must be positive. Not sure if that does anything funky, but we'll see!

STAN for (f)

```
data {
  int<lower=0> N; //Number of Schools
  int<lower=0> n; //Total datapoints
  array[n] real y; // Responses
  array[n] int grp; //Associated group for each response
  real sigma2_0;
  real eta_0;
  real tau2_0;
  real mu_0;
  real gamma2_0;
  real alpha_0;
}

parameters {
  array[N] real theta;
  array[N] real<lower=0> sigma2;
  real mu;
  real<lower=0> nu;
  real<lower=0> tau2;
}

transformed parameters {
  array[n] real theta_vec;
  array[n] real sigma2_vec;
  for (i in 1:n) {
    theta_vec[i] = theta[grp[i]];
  }
  for (i in 1:n) {
    sigma2_vec[i] = sigma2[grp[i]];
  }
}

model {
  y ~ normal(theta_vec, sqrt(sigma2_vec));
  theta ~ normal(mu, sqrt(tau2));
  sigma2 ~ inv_gamma(nu/2, nu * sigma2_0/2);
  tau2 ~ inv_gamma(eta_0/2, eta_0 * tau2_0/2);
  mu ~ normal(mu_0, sqrt(gamma2_0));
}
```

```

    nu ~ exponential(alpha_0);
  }

generated quantities {
  vector[n] y_rep;
  vector[n] y_upc;
  vector[n] y_llik;
  vector[N] theta_upc;
  vector[N] sigma2_upc;

  for (i in 1:n) {
    real theta_i = theta[grp[i]];
    real sigma2_i = sigma2[grp[i]];
    y_rep[i] = normal_rng(theta_i, sqrt(sigma2_i));
    y_upc[i] = normal_cdf(y[i] | theta_i, sqrt(sigma2_i));
    y_llik[i] = normal_lpdf(y[i] | theta_i, sqrt(sigma2_i));
  }

  for (i in 1:N) {
    theta_upc[i] = normal_cdf(theta[i] | mu, sqrt(tau2));
  }

  for (i in 1:N) {
    sigma2_upc[i] = inv_gamma_cdf(sigma2[i] | nu/2, nu * sigma2_0 / 2);
  }
}

```

```

alpha_0 <- .5
data_stan_2 <- list(N = N, n = n, y = dt$times, grp = dt$school, sigma2_0 = sigma2_0, eta_0 = eta_0)

params <- c("theta", "mu", "sigma2", "tau2", "y_rep", "y_upc", "y_llik", "theta_upc", "sigma2_upc")

fit_stan_2 <- stan(model_code = readLines("q1_f.stan"),
                  data = data_stan_2, pars = params,
                  iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan_2 <- extract(fit_stan_2) # collect samples from all chains

n_sims2 <- nrow(sim_stan_2$theta)
var_diffs2 <- numeric(n_sims2)
for (i in 1:n_sims2) {
  var_diffs2[[i]] <- diff(range(tapply(sim_stan_2$y_rep[i, ], dt$school, var)))
}

obs_var_diff_2 <- diff(range(tapply(dt$times, dt$school, var)))
var_diff_p_2 <- mean(var_diffs2 > obs_var_diff_2)

```

```
loo2 <- loo::loo(sim_stan_2$y_lik)
loo2_est <- round(loo2$estimates[1, 1], 3)
loo2_params <- round(loo2$estimates[2, 1], 2)
waic2 <- loo::waic(sim_stan_2$y_lik)
```

Warning:

1 (0.6%) p_waic estimates greater than 0.4. We recommend trying loo instead.

```
waic2_est <- round(waic2$estimates[1, 1], 3)
waic2_params <- round(waic2$estimates[2, 1], 2)
```

```
upc_pvals_y_2 <- apply(sim_stan_2$y_upc, 1, function(x) goftest::ad.test(x, null = "punif")$p)
cauchy_y_p_2 <- round(pcauchy(mean(tan(pi * (0.5 - upc_pvals_y_2))), lower.tail = FALSE), 4)
```

```
upc_pvals_theta_2 <- apply(sim_stan_2$theta_upc, 1, function(x) goftest::ad.test(x, null = "punif")$p)
cauchy_theta_p_2 <- round(pcauchy(mean(tan(pi * (0.5 - upc_pvals_theta_2))), lower.tail = FALSE), 4)
```

This model gives a LOO of -501.795 and a WAIC of -501.697. The models give model parameter estimates of 10.56 (LOO) and 10.47 (WAIC).

g)

Comparing the two models, the second model has slightly lower LOOIC and WAIC which imply they are somewhat better. However, performing the same UPC measures as with the first model, we get a y_{UPC} p-value of 0.2251 which is non-significant, especially when compared to the first model. The θ_{UPC} p-value is 0.399 which is approximately the same as the first model. Furthermore, when performing the same Predictive Performance metric (range of group means), we get a p-value of 0.9215556 for the second model, which is much higher than the first one.

Due to these metrics, I think I would still choose to go with the first model, primarily because of the strong evidence from the predictive performance measure which indicates that we are not properly capturing the behavior of the data in the second model.

2

$$y_i = \alpha + x_i^\top \beta + \epsilon_i; \epsilon \sim N(0, 1) | x_{i,j} \sim N(0, 1); i = 1, \dots, n; j = 1, \dots, p; p = 10; \beta = (10, -5, 3, 0, 0, 0, 0, 0, 0, 0)$$

```
# Generate sample data
set.seed(1337)
beta_true <- c(10, -5, 3, 0, 0, 0, 0, 0, 0, 0)
alpha_true <- 0
n <- 100
p <- 10
```

```
X <- matrix(rnorm(n * p), nrow = n, ncol = p)
y <- alpha_true + X %*% beta_true + rnorm(n, 0, 1)
```

a)

Assume σ^2 , the variance of ϵ_i is unknown and include in the model. I will use a conjugate prior of Normal-Inverse-Gamma where $\beta|\sigma^2 \sim N(0, \sigma^2 I); \sigma^2 \sim \text{Inv-Gamma}(\alpha_0, \beta_0)$

STAN code:

```
data {
  int<lower=0> n; //Total datapoints
  int<lower=0> p; // Total betas
  matrix[n, p] X; // Predictor matrix
  vector[n] y; // Responses
  real alpha_0;
  real beta_0;
  real lambda_0; // scale factor
}

parameters {
  vector[p] beta;
  real alpha;
  real<lower=0> sigma2;
}

model {
  alpha ~ normal(0, 10); // Weekly informative prior
  sigma2 ~ inv_gamma(alpha_0, beta_0);
  beta ~ normal(0, sigma2 * lambda_0);
  y ~ normal(X * beta + alpha, sqrt(sigma2)); // data model
}

generated quantities {
  vector[n] y_llik;

  for (i in 1:n) {
    y_llik[i] = normal_lpdf(y[i] | X[i] * beta + alpha, sqrt(sigma2));
  }
}
```

```
alpha_0 <- 2
beta_0 <- 2
mu_0 <- 2
lambda_0 <- 1
data_stan <- list(n = n, p = p, X = X, y = y[,1], alpha_0 = alpha_0, beta_0 = beta_0, lambda_0 = lambda_0)
```



```

params <- c("beta", "alpha", "sigma2")

fit_stan_a <- stan(model_code = readLines("q2_a.stan"),
                  data = data_stan, pars = params,
                  iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan_a <- extract(fit_stan_a) # collect samples from all chains

```

b)

STAN Code:

```

data {
  int<lower=0> n; //Total datapoints
  int<lower=0> p; // Total betas
  matrix[n, p] X; // Predictor matrix
  vector[n] y; // Responses
  real alpha_0;
  real beta_0;
  real lambda_0; // scale factor
}

parameters {
  vector[p] beta_raw;
  real alpha;
  real<lower=0> sigma2;
  vector<lower=0>[p] lambda;
  real<lower=0> tau;
}

transformed parameters {
  vector[p] beta;
  beta = beta_raw .* lambda * tau * sqrt(sigma2); // Multiply by tau
}

model {
  lambda ~ cauchy(0, 1);
  tau ~ cauchy(0, 1);
  alpha ~ normal(0, 10); // Weekly informative prior
  sigma2 ~ inv_gamma(alpha_0, beta_0);
  beta_raw ~ std_normal();
  y ~ normal(X * beta + alpha, sqrt(sigma2)); // data model
}

```

```

params <- c("beta", "alpha", "sigma2", "lambda", "tau")

fit_stan_b <- stan(model_code = readLines("q2_b.stan"),
                  data = data_stan, pars = params,
                  iter = 5000, warmup = 500, thin = 5, chains = n_cores)

```

Warning in readLines("q2_b.stan"): incomplete final line found on 'q2_b.stan'

Warning: There were 386 divergent transitions after warmup. See <https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup> to find out why this is a problem and how to eliminate them.

Warning: There were 1949 transitions after warmup that exceeded the maximum treedepth. Increase <https://mc-stan.org/misc/warnings.html#maximum-treedepth-exceeded>

Warning: Examine the pairs() plot to diagnose sampling problems

```

sim_stan_b <- extract(fit_stan_b) # collect samples from all chains

```

c)

```

sim_stan_a$beta[1,]

```

```

[1]  9.9877157953 -5.0618751135  3.2805449540  0.5759323332  0.0006269427
[6]  0.0034530155 -0.0258284885 -0.3358156497 -0.0293221864  0.1804325273

```

```

a_beta_means <- numeric(p)

a_beta_means <- round(apply(sim_stan_a$beta, 2, mean), 3)
a_beta_quants <- round(apply(sim_stan_a$beta, 2, quantile, probs = c(0.025, 0.975)), 3)
b_beta_means <- round(apply(sim_stan_b$beta, 2, mean), 3)
b_beta_quants <- round(apply(sim_stan_b$beta, 2, quantile, probs = c(0.025, 0.975)), 3)

beta_vals <- c(1:10)
tab <- as.data.frame(cbind(beta_vals, a_beta_means, a_beta_quants[1, ], a_beta_quants[2, ], b_beta_means, b_beta_quants[1, ], b_beta_quants[2, ]))
colnames(tab) <- c('Beta', 'Model 1 Mean', 'Model 1 Lower', 'Model 1 Upper', 'Model 2 Mean', 'Model 2 Lower', 'Model 2 Upper')
kable(tab)

```

Beta	Model 1 Mean	Model 1 Lower	Model 1 Upper	Model 2 Mean	Model 2 Lower	Model 2 Upper
1	9.992	9.731	10.258	10.037	9.845	10.227

Beta	Model 1 Mean	Model 1 Lower	Model 1 Upper	Model 2 Mean	Model 2 Lower	Model 2 Upper
2	-4.958	-5.258	-4.657	-4.986	-5.197	-4.777
3	2.997	2.723	3.276	3.018	2.829	3.207
4	0.235	-0.087	0.560	0.172	-0.025	0.409
5	0.086	-0.173	0.341	0.060	-0.077	0.243
6	0.012	-0.258	0.280	0.014	-0.137	0.175
7	-0.056	-0.349	0.243	-0.034	-0.219	0.124
8	0.066	-0.228	0.357	0.041	-0.110	0.228
9	-0.017	-0.316	0.281	-0.007	-0.177	0.155
10	0.198	-0.077	0.484	0.137	-0.032	0.346

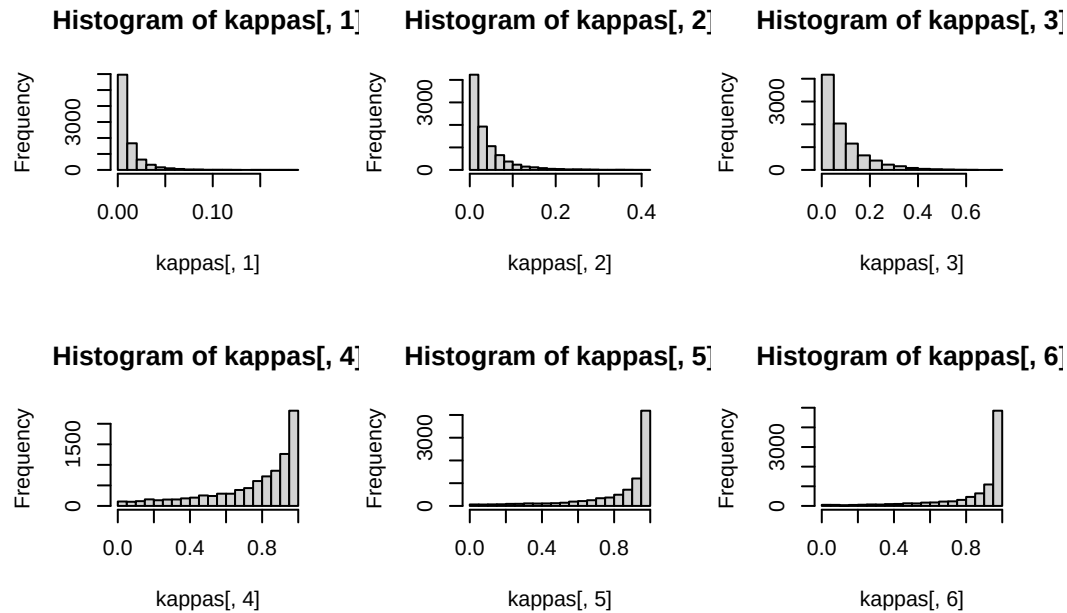
The intervals are consistently smaller on the second model, which implies it's a little bit better.

d)

I plot the distributions of κ for the first 6 κ 's (the other 4 are similar to the previous)

```
n_sims <- nrow(sim_stan_b$sigma2)
kappas <- matrix(NA, nrow = n_sims, ncol = p)
for (i in 1:n_sims) {
  kappas[i,] <- 1/(1 + sim_stan_b$lambda[i,]^2 * sim_stan_b$tau[i])
}

par(mfrow = c(2, 3))
hist(kappas[,1])
hist(kappas[,2])
hist(kappas[,3])
hist(kappas[,4])
hist(kappas[,5])
hist(kappas[,6])
```



```
par(mfrow = c(1, 1))
```

This shows that yes, this model does appear to correctly include the first 3 parameters, and shrink away the rest.

e)

```
# Generate sample data
set.seed(1337)
beta_true <- c(10, -5, 3, 0, 0, 0, 0, 0, 0, 0)
alpha_true <- 0
n <- 100
p <- 10
sigma <- runif(p)
X <- matrix(rnorm(n * p), nrow = n, ncol = p)
y <- alpha_true + X %*% beta_true + rnorm(n, 0, sigma)
```

STAN code for this given previously

```
alpha_0 <- 2
beta_0 <- 2
mu_0 <- 2
lambda_0 <- 1
data_stan <- list(n = n, p = p, X = X, y = y[,1], alpha_0 = alpha_0, beta_0 = beta_0, lambda_0 = lambda_0)

params <- c("beta", "alpha", "sigma2", "y_lik")
```

```

fit_stan_e_1 <- stan(model_code = readLines("q2_a.stan"),
                    data = data_stan, pars = params,
                    iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan_e_1 <- extract(fit_stan_e_1) # collect samples from all chains

loo1 <- loo::loo(sim_stan_e_1$y_lik)
loo1_est <- round(loo1$estimates[1, 1], 3)
loo1_params <- round(loo1$estimates[2, 1], 2)
waic1 <- loo::waic(sim_stan_e_1$y_lik)
waic1_est <- round(waic1$estimates[1, 1], 3)
waic1_params <- round(waic1$estimates[2, 1], 2)

```

STAN Code:

```

data {
  int<lower=0> n; //Total datapoints
  int<lower=0> p; // Total betas
  matrix[n, p] X; // Predictor matrix
  vector[n] y; // Responses
  real alpha_0;
  real beta_0;
  real lambda_0; // scale factor
}

parameters {
  vector[p] beta;
  real alpha;
  real tau2;
  vector<lower=0>[n] sigma2;
}

model {
  alpha ~ normal(0, 10); // Weekly informative prior
  sigma2 ~ inv_gamma(alpha_0, beta_0);
  beta ~ normal(0, tau2 * lambda_0);
  y ~ normal(X * beta + alpha, sqrt(sigma2)); // data model
}

generated quantities {
  vector[n] y_lik;

  for (i in 1:n) {
    y_lik[i] = normal_lpdf(y[i] | X[i] * beta + alpha, sqrt(sigma2[i]));
  }
}

```

```

fit_stan_e_2 <- stan(model_code = readLines("q2_e.stan"),
                    data = data_stan, pars = params,
                    iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan_e_2 <- extract(fit_stan_e_2) # collect samples from all chains

loo2 <- loo::loo(sim_stan_e_2$y_lik)
loo2_est <- round(loo2$estimates[1, 1], 3)
loo2_params <- round(loo2$estimates[2, 1], 2)
waic2 <- loo::waic(sim_stan_e_2$y_lik)
waic2_est <- round(waic2$estimates[1, 1], 3)
waic2_params <- round(waic2$estimates[2, 1], 2)

```

Model1 gives a LOO of -128.069 and a WAIC of -128.044. Model2 gives a LOO of -114.448 and a WAIC of -114.323. Both metrics are lower for the second model (the unequal variances), indicating that it is perhaps a better fit.

Acknowledgements

Estimated time

I estimate this assignment took me in total around 8 hours to complete.

Resources Used

I used GoogleAI to help me understand what the heck to do with Posterior Predictive Checks (ie. how to generate the y_{rep} , and how to compare that to y_{obs}), as well as debugging my RStan code, and particularly how to do the various priors for the second question.